2ndOpinionMD — Timeline Upload Guide

For Dr. Kaleb Michaud, PhD · FORWARD Registry Integration

Dylan McCapes · 2ndOpinionMD

April 2026

1. Overview

2ndOpinionMD accepts patient timelines via a single HTTP endpoint and runs them through our local **eoh-llama 8B** model for structured event extraction, clinical arc identification, and knowledge graph construction. All inference runs on-premise on PortalNode hardware (RTX 4090 GPUs) — no patient data leaves the local network.

The endpoint supports two upload formats:

Format	Best for	Content
PDF (default)	Patients uploading their medical records directly	Unencrypted PDF of any size
JSON	Structured EHR exports from FORWARD or other registries	Array of event objects

Both formats are chunked automatically to fit within the model's context window. Progress streams back in real time as Server-Sent Events (SSE).

---

2. Endpoint

```
POST /api/timeline/{patient_id}/infer
Content-Type: multipart/form-data
```

Base URL: `https://2ndopinionmd.ai`

Replace `{patient_id}` with a unique, de-identified patient identifier (e.g. `forward_patient_00142`).

---

3. Uploading a PDF (Easiest)

This is the simplest path. The patient (or data coordinator) uploads an unencrypted PDF of their medical records. The server extracts text from every page, chunks the pages into batches, and runs each batch through the 8B model.

curl example

```
curl -N https://2ndopinionmd.ai/api/timeline/forward_patient_00142/infer \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -F "file=@/path/to/patient_timeline.pdf" \
  -F "store_results=true" \
  -F "build_graph=true"
```

Form fields

Field	Required	Default	Description
`file`	Yes	—	The PDF file
`format`	No	`pdf`	Set to `pdf` (or omit entirely)
`password`	No	—	PDF password, if the file is encrypted
`store_results`	No	`true`	Write extracted events to the database
`build_graph`	No	`true`	Build a PatientTimelineVision knowledge graph
`model`	No	`eoh-llama3.1:8b`	Ollama model name
`num_ctx`	No	`32768`	Context window size (tokens)
`question`	No	*(clinical investigation)*	Override the extraction prompt

What happens

1. **Upload** — File bytes are received (progress visible in curl).
2. **PDF extraction** — pypdf extracts text from every page in a background thread. An SSE `status` event reports progress.
3. **PII scrub** — Extracted text is scrubbed of personally identifiable information (MRN, SSN, DOB, phone/fax numbers, email addresses, street addresses, and detected patient names). Clinically relevant fields (e.g. sex) are preserved. An SSE `pii_scrub_done` event confirms completion.
4. **Heuristic pre-scan** — A fast regex pass (~0.5 ms/page) runs over the scrubbed pages, pulling dates, medications (with dose and route), lab results, diagnoses, ICD-10 codes, and section context. Pre-scan events are immediately added to the knowledge graph and temporal connascence edges (events within 7 days) are auto-linked. An SSE `pre_scan_done` event reports what was found.
5. **Batching** — Pages are grouped into batches that fit the 8B model's context window (~48k characters, up to 10 pages per batch). Each batch includes a "pre-scan skeleton" showing what regex already extracted.
6. **Inference** — Each batch is sent to eoh-llama 8B. The model verifies the skeleton, corrects any errors, and supplements with events regex cannot find (symptoms, flares, treatment responses, clinical reasoning, causal relationships). LLM output is scrubbed a second time to catch any PII fragments the model may echo. SSE events report per-batch progress.
7. **Graph construction** — Scrubbed, LLM-extracted events merge into the pre-scan graph. A final temporal connascence pass runs over the complete graph.
8. **Complete** — A final SSE event reports totals.

Processing time estimates

Benchmarked on PortalNode-01 (RTX 4090, eoh-llama 8B, 32k context):

Input	Size	Batches	Events extracted	Wall clock
PDF (Kaiser full record)	4,223 pages / 177 MB	423	3,556	~2.1 hours
PDF (typical visit record)	~50 pages	~5	~40–80	~2–4 min
PDF (specialist referral)	~10 pages	1	~10–20	~30 sec

Rule of thumb for PDFs: ~30 seconds per batch, ~10 pages per batch. Divide the page count by 10, multiply by 30 seconds.

Notes on large PDFs

- PDFs of any size are supported (tested with 4,223-page, 177 MB records).
- The heuristic pre-scan completes in seconds even for very large PDFs (~2 seconds for 4,000 pages) and immediately populates the graph.
- Only one inference can run at a time (GPU constraint). A second request returns HTTP 409 with details about the active job.

---

4. Uploading Structured EHR JSON

If the patient's data is already structured (e.g. a FORWARD registry export or FHIR-formatted EHR dump), upload it as JSON. This skips PDF extraction entirely and goes straight to batching and inference.

curl example

```
curl -N https://2ndopinionmd.ai/api/timeline/forward_patient_00142/infer \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -F "file=@/path/to/patient_ehr_export.json" \
  -F "format=json" \
  -F "store_results=true" \
  -F "build_graph=true"
```

The only difference from PDF is adding `-F "format=json"`.

Expected JSON structure

The file should contain a JSON array of event objects, or an object with an `events` key:

```
{
  "events": [
    {
```

```
      "ts": "2024-01-15T10:30:00Z",
      "event_type": "lab",
      "source": "EHR",
      "text": "CRP elevated at 38.2 mg/L...",
      "structured": {
        "CRP": 38.2,
        "CRP_unit": "mg/L",
        "ESR": 54
      }
    },
    {
      "ts": "2024-03-20T14:00:00Z",
      "event_type": "visit",
      "source": "EHR",
      "text": "Follow-up visit. 3 swollen joints...",
      "structured": {
        "swollen_joint_count": 3,
        "das28_crp": 4.2
      }
    }
  ]
}
```

Event fields

Field	Required	Description
`ts`	Recommended	ISO-8601 timestamp
`event_type`	No	One of: `lab`, `symptom`, `medication`, `imaging`, `flare`, `visit`, `procedure`, `diagnosis`, `note`
`source`	No	Data source label (e.g. `EHR`, `FORWARD`, `patient_upload`)
`text`	Recommended	Free-text narrative describing the event
`structured`	No	Key-value pairs for extracted values (lab results, scores, etc.)
`meta`	No	Arbitrary metadata

A bare JSON array (without the `events` wrapper) is also accepted:

```
[
  { "ts": "2024-01-15T10:30:00Z", "event_type": "lab", "text": "..." },
  { "ts": "2024-03-20T14:00:00Z", "event_type": "visit", "text": "..." }
]
```

Processing time estimates for JSON

Events are serialized to text blocks and grouped into batches the same way PDF pages are. JSON skips PDF extraction entirely, so it starts inference faster.

Input	Events	Batches (approx)	Wall clock
Small study export	~30–50	1–2	~30–60 sec
FORWARD registry (one patient, years)	~200–500	5–15	~3–8 min
Large longitudinal EHR dump	~2,000+	15–25	~8–12 min

Rule of thumb for JSON: each batch holds ~48k characters of serialized events (~80–150 events depending on text density). Divide event count by 100, multiply by 30 seconds.

---

5. Monitoring Progress

The endpoint returns a `text/event-stream` (Server-Sent Events). Each event has a type and a JSON payload:

```
event: accepted
data: {"patient_id":"...","model":"eoh-llama3.1:8b","file":"timeline.pdf",...}

event: status
data: {"phase":"pdf_extracting","message":"Extracting text from PDF (50 MB)..."}

event: pdf_read
data: {"total_pages":1847,"pages_with_text":1623,"total_chars":4200000}

event: status
data: {"phase":"pre_scan","message":"Running heuristic pre-scan on 1623 pages..."}

event: pre_scan_done
data: {"events":412,"dates":1847,"meds":38,"labs":95,"dx":67,"temporal_edges":280,
       "graph_events":412}

event: infer_start
data: {"patient_id":"...","total_batches":162,"model":"eoh-llama3.1:8b",...}

event: batch_start
data: {"batch":1,"total":162,"chars":47200,"page_range":"1-10"}

event: batch_done
data: {"batch":1,"extracted":18,"stored":18,"elapsed_ms":23000}

event: graph_update
data: {"total_events":430,"total_edges":285}

... (repeats for each batch) ...

event: complete
data: {"batches_processed":162,"events_extracted":892,"total_elapsed_ms":1500000,...}
```

The `-N` flag in curl disables output buffering so events appear in real time.

Checking status

To check if an inference is already running:

```
curl https://2ndopinionmd.ai/api/timeline/infer/status
```

Returns:

```
{"busy": true, "active_job": {"patient_id": "...", "started_at": "..."}}
```

or `{"busy": false}` if the GPU is available.

---

6. FORWARD Convenience Endpoint

For initial integration testing, a dedicated endpoint is available that does not require a patient ID — it uses a fixed de-identified ID (`forward_patient_00142`) and an RA-specific extraction prompt.

This endpoint requires a bearer token. The token was delivered to you via Signal. Include it in every request as shown below.

Upload a PDF (simplest)

```
curl -N https://2ndopinionmd.ai/api/timeline/forward/upload \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -F "file=@patient_timeline.pdf"
```

Upload EHR JSON

```
curl -N https://2ndopinionmd.ai/api/timeline/forward/upload \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -F "file=@patient_ehr_export.json" \
  -F "format=json"
```

Both default to `store_results=true` and `build_graph=true`.

The general endpoint (`/api/timeline/{patient_id}/infer`) is also available if you need to specify patient IDs per upload.

> **Token delivery**: Your API token was sent via Signal with a disappearing-message timer. If you need the token re-issued, contact Dylan via Signal. Do not store the token in shared documents, email, or version control.

---

7. Querying the Graph

Once a timeline has been ingested, the knowledge graph is available for queries via the `/api/graph/` endpoints. All endpoints use the same `patient_id` used during upload (e.g. `forward_patient_00142`).

All `forward_*` graph queries require the same bearer token used for upload. Include `-H "Authorization: Bearer YOUR_TOKEN_HERE"` in every request.

Full graph export

```
curl -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  https://2ndopinionmd.ai/api/graph/forward_patient_00142/full -o graph.json
```

Returns the complete knowledge graph as JSON — every event, edge, arc, and metadata field. Use this to pull the entire dataset for downstream analysis, visualisation, or import into another system.

Graph snapshot (lightweight overview)

```
curl -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  https://2ndopinionmd.ai/api/graph/forward_patient_00142/snapshot | python3 -m json.tool
```

Returns event counts by type, date ranges, and a compact node list.

Structural topology

```
curl -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  https://2ndopinionmd.ai/api/graph/forward_patient_00142/topology
```

Connected components, orphan events, hub events (most connected), temporal gaps (>90 days), edge type distribution, and graph density.

Search events

```
# All medications
curl -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  "https://2ndopinionmd.ai/api/graph/forward_patient_00142/events?event_type=medication"

# Search by keyword
curl -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  "https://2ndopinionmd.ai/api/graph/forward_patient_00142/events?q=methotrexate"

# Filter by date range
curl -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  "https://2ndopinionmd.ai/api/graph/forward_patient_00142/events?date_from=2020-01-01&date_to=
```

Single event + neighbours

```
curl -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  https://2ndopinionmd.ai/api/graph/forward_patient_00142/event/dx_001
```

Returns the event detail and all connected events grouped by edge type.

Priority traversal

```
curl -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  "https://2ndopinionmd.ai/api/graph/forward_patient_00142/traverse?seed=dx_001&max_nodes=50"
```

Walks the graph from a seed event following highest-value edges first (causal > diagnostic > treatment > drug_response > lab_trend > temporal).

Temporal gaps

```
curl -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  "https://2ndopinionmd.ai/api/graph/forward_patient_00142/gaps?min_gap_days=60"
```

Periods of silence — often indicating lost-to-follow-up, care transitions, or extraction gaps.

Negative space

```
curl -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  https://2ndopinionmd.ai/api/graph/forward_patient_00142/negative
```

Expected-but-absent patterns: diagnoses without follow-up labs, medications without efficacy assessments — the gaps where diagnostic mysteries hide.

Ask a question (LLM-powered)

```
curl -X POST -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  https://2ndopinionmd.ai/api/graph/forward_patient_00142/ask \
  -d '{"question": "What biologic switches has this patient had and why?"}'
```

Sends the question to eoh-llama 8B with graph context and returns a cited answer.

Edge list

```
# All edges
curl -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  https://2ndopinionmd.ai/api/graph/forward_patient_00142/edges

# Only causal edges
curl -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  "https://2ndopinionmd.ai/api/graph/forward_patient_00142/edges?kind=causal"
```

---

8. Quick Reference

FORWARD endpoint (recommended for testing — requires bearer token)

```
# PDF
curl -N -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  https://2ndopinionmd.ai/api/timeline/forward/upload \
  -F "file=@TIMELINE.pdf"

# JSON
curl -N -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  https://2ndopinionmd.ai/api/timeline/forward/upload \
```

```
  -F "file=@EHR_EXPORT.json" \
  -F "format=json"
```

General endpoint (per-patient ID)

```
# PDF
curl -N https://2ndopinionmd.ai/api/timeline/PATIENT_ID/infer \
  -F "file=@TIMELINE.pdf"

# JSON
curl -N https://2ndopinionmd.ai/api/timeline/PATIENT_ID/infer \
  -F "file=@EHR_EXPORT.json" \
  -F "format=json"
```

Check GPU status

```
curl https://2ndopinionmd.ai/api/timeline/infer/status
```

---

9. Security and Privacy

- **Bearer token authentication**: The FORWARD upload endpoint and all `forward_*` graph queries require a bearer token. Tokens are delivered via Signal with disappearing-message timers. Constant-time comparison prevents timing-based attacks.
- **PII scrubbing**: All uploaded data passes through an automatic PII scrubber before LLM processing and graph storage. MRN, SSN, DOB, phone/fax numbers, email addresses, and street addresses are redacted. Clinically relevant fields (e.g. sex) are preserved.
- All inference runs locally on PortalNode hardware. No data is sent to external APIs.
- Patient identifiers should be de-identified before upload (use study IDs, not names or MRNs).
- The eoh-llama 8B model runs entirely on-premise via Ollama.
- Uploaded files are held in memory during processing and are not persisted to disk.
- Extracted events are stored in PostgreSQL only if `store_results=true`.

Questions or issues: dylan@2ndopinionmd.ai